FRAUDSHIELD AI — REAL-TIME FINANCIAL FRAUD DETECTION SYSTEM

By Pendyala Pradeep Reddy

Under the guidance of Chandan Mishra

CONTENTS

Chapter	Title	Page
1	**Introduction**	1
1.1	Introduction	1
1.2	Objectives	2
1.3	Motivation	3
1.4	Scope of the Work	4
1.5	Feasibility Study	5
2	**Design and Implementation**	6
2.1	Introduction	6
2.2	Requirement Gathering	7
2.2.1	Functional Requirements	7
2.2.2	Non-Functional Requirements	8
2.3	Proposed Design	8
2.3.1	System Architecture	9
2.3.2	Snapshots of Dashboard and Visualizations	9
2.4	Hardware Requirements	16
2.5	Software Requirements	17
3	**Results and Discussion**	18
3.1	Introduction	18
3.2	Dataset Overview	18
3.3	Training and Evaluation	19
3.3.1	Preprocessing and Feature Engineering	19
3.3.2	Handling Imbalance	19

3.3.3	Models	19
3.3.4	Evaluation Strategy	19
3.4	Model Performance	20
3.5	Explainability with SHAP	20
3.6	Discussion and Key Findings	20
3.7	Observations	20
4	**Conclusion**	21
4.1	Conclusion	21
4.2	Future Scope	21
	References	22

CHAPTER 1

INTRODUCTION

1.1 Introduction

Financial fraud is a major concern in modern digital banking and online payment systems. Every year, billions of dollars are lost due to fraudulent credit card transactions. Fraud not only affects financial institutions but also creates serious problems for customers, such as unauthorized charges, frozen accounts, and identity theft.

Traditional rule-based fraud detection systems are limited because they rely on predefined patterns. Fraudsters constantly change their techniques, making it difficult for static rules to detect new types of fraud. Machine learning offers a more adaptive solution because models can learn patterns from historical transaction data and identify suspicious behavior automatically.

FraudShield AI is a machine learning based fraud detection system designed to detect suspicious credit card transactions. The system processes transaction data, trains multiple machine learning models, and provides predictions through an interactive dashboard.

The project goes beyond simple model training. It includes a complete pipeline that handles preprocessing, model training, model evaluation, explainability, and real-time simulation. The system also provides alerts and visual analytics to help analysts understand fraud patterns.

The final system includes a multi-page Streamlit dashboard that allows users to explore the dataset, compare models, simulate transactions, view explanations using SHAP values, and monitor fraud alerts.

1.2 Objectives

The main objectives of this project are listed below.

1. Load and preprocess the Kaggle credit card fraud dataset containing 284,807 transactions.

2. Apply feature scaling to the Time and Amount variables using RobustScaler.

3. Handle extreme class imbalance using SMOTE oversampling applied only to the training dataset.

4. Train multiple machine learning models including Logistic Regression, Random Forest, XGBoost, and Isolation Forest.

5. Evaluate models using metrics such as Accuracy, Precision, Recall, F1 Score, ROC-AUC, and PR-AUC.

6. Develop an interactive Streamlit dashboard for visualization and monitoring.

7. Provide real-time transaction simulation for fraud prediction.

8. Integrate SHAP based explainability to interpret model decisions.

9. Implement alert systems through Email and WhatsApp notifications.

10. Add user authentication so that multiple analysts can securely access the system.

1.3 Motivation

Fraud detection is an important application of machine learning because financial institutions lose large amounts of money due to fraudulent activities. Even a small improvement in fraud detection accuracy can result in significant financial savings.

Another motivation for this project is the gap between academic demonstrations and practical applications. Many fraud detection projects stop at model training and evaluation. However, real systems require additional components such as user interfaces, alerts, explainability, and authentication.

This project focuses on building a system that resembles a real monitoring tool rather than just a research experiment. The dashboard allows users to interact with the data, simulate transactions, and understand why a transaction was classified as fraudulent.

1.4 Scope of the Work

The scope of this project includes the following components.

Data ingestion from the Kaggle credit card dataset.

Data preprocessing including scaling, feature selection, and class balancing.

Training and evaluation of multiple machine learning models.

Interactive data visualization using Plotly charts.

Development of a multi-page Streamlit dashboard.

Integration of SHAP based explainability for model interpretation.

Real-time transaction simulation.

Fraud alert notifications through email and WhatsApp.

User authentication for secure access.

The project does not include full cloud deployment, real-time streaming pipelines, or integration with live banking systems. However, the system design allows future extension toward those capabilities.

1.5 Feasibility Study

The feasibility of the project was analyzed in terms of data availability, tools, computation resources, and time.

Data availability:
The project uses the publicly available Kaggle Credit Card Fraud Detection dataset. The dataset contains 284,807 anonymized transactions recorded over two days.

Tools:
The system is implemented using the Python ecosystem. Libraries such as Pandas, NumPy, scikit-learn, XGBoost, SHAP, Plotly, and Streamlit provide reliable tools for data analysis, machine learning, and visualization.

Computation:
The project can run on a standard laptop with 8 GB RAM. Model artifacts are cached using joblib so that training is required only once.

Time feasibility:
Machine learning libraries provide built-in implementations of algorithms, which reduces development time. The main development effort focused on dashboard design and system integration.

CHAPTER 2

DESIGN AND IMPLEMENTATION

2.1 Introduction

This chapter explains the system architecture, system requirements, and the implementation approach used in the project. The system is designed in a modular way so that each component performs a specific task.

Modules include preprocessing, model training, explainability, alerts, authentication, and dashboard visualization.

2.2 Requirement Gathering

2.2.1 Functional Requirements

The system should load the credit card dataset in CSV format.

The system should preprocess the dataset and scale the Time and Amount features.

The system should perform train test splitting with stratified sampling.

The system should handle class imbalance using SMOTE.

The system should train four machine learning models.

The system should evaluate models using multiple performance metrics.

The system should allow users to simulate transactions and obtain predictions.

The system should generate SHAP based explainability plots.

The system should provide email and WhatsApp alerts when fraud is detected.

The system should include a user authentication system.

2.2.2 Non Functional Requirements

Accuracy: The models should maintain high recall to detect fraudulent transactions.

Scalability: The system should handle the full dataset without performance issues.

Usability: The dashboard must be simple to navigate for analysts.

Performance: After training, the system should load quickly using cached models.

Security: Passwords must be stored using secure hashing.

Maintainability: Code must be modular and easy to modify.

2.3 Proposed Design

The system is divided into separate modules stored in the src directory.

Each module performs a specific function such as preprocessing, modeling, explainability, alerts, or authentication.

2.3.1 System Architecture

```
DA2/
app.py
creditcard.csv
requirements.txt
users.json

src/
preprocessing.py
models.py
explainability.py
utils.py
alerts.py
auth.py

cache/
scaler.pkl
splits.pkl
models.pkl
```

Data Flow

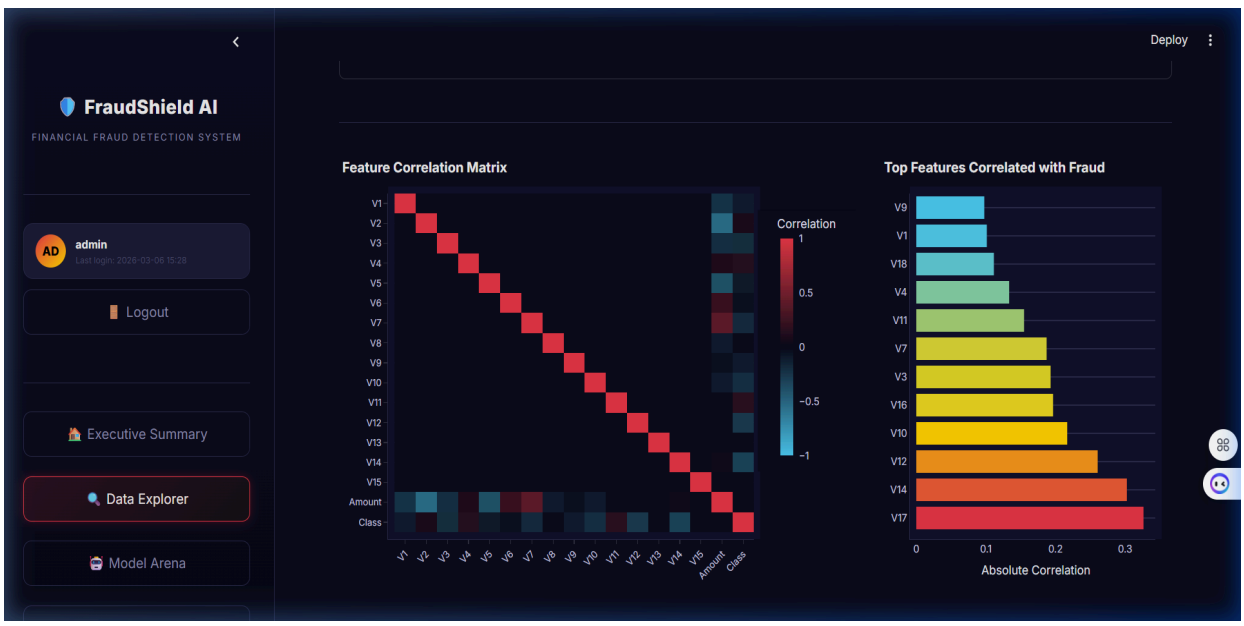
1. Dataset is loaded using Pandas.
2. Time and Amount features are scaled using RobustScaler.
3. Train test split is performed using stratified sampling.
4. SMOTE is applied to balance the training dataset.
5. Models are trained and evaluated.
6. SHAP explainability is generated.
7. Dashboard displays results and predictions.
8. Alert module sends notifications.

2.3.2 Snapshots of Dashboard and Visualizations

01_executive_summary

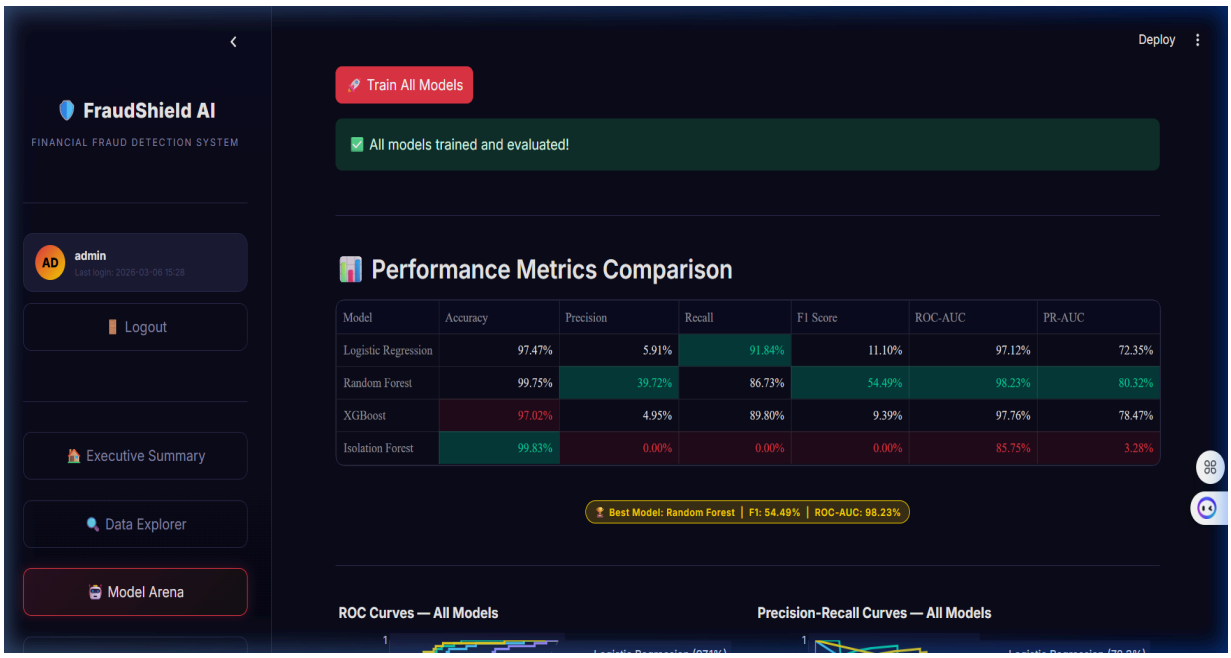
	Time	V1	V2	V3	V4	V5	V6	V7	V8	V9	V10
count	284807.0000	284807.0000	284807.0000	284807.0000	284807.0000	284807.0000	284807.0000	284807.0000	284807.0000	284807.0000	284807
mean	94813.8596	0.0000	0.0000	-0.0000	0.0000	0.0000	0.0000	-0.0000	0.0000	-0.0000	0
std	47488.1460	1.9587	1.6513	1.5163	1.4159	1.3802	1.3323	1.2371	1.1944	1.0986	1
min	0.0000	-56.4075	-72.7157	-48.3256	-5.6832	-113.7433	-26.1605	-43.5572	-73.2167	-13.4341	-24
25%	54201.5000	-0.9204	-0.5985	-0.8904	-0.8486	-0.6916	-0.7683	-0.5541	-0.2086	-0.6431	-0
50%	84692.0000	0.0181	0.0655	0.1798	-0.0198	-0.0543	-0.2742	0.0401	0.0224	-0.0514	-0
75%	139320.5000	1.3156	0.8037	1.0272	0.7433	0.6119	0.3986	0.5704	0.3273	0.5971	0
max	172792.0000	2.4549	22.0577	9.3826	16.8753	34.8017	73.3016	120.5895	20.0072	15.5950	23

02_data_explorer



02_data_explorer_charts

03_model_arena



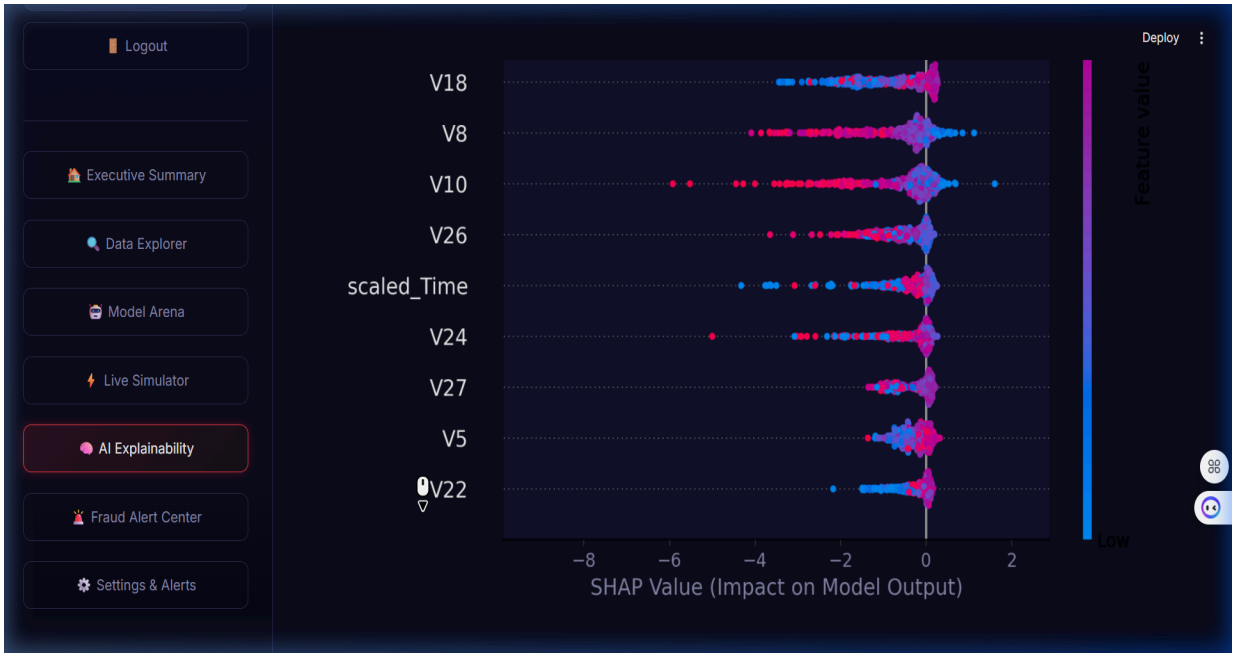
Model	Accuracy	Precision	Recall	F1 Score	ROC-AUC	PR-AUC
Logistic Regression	97.47%	5.91%	91.84%	11.10%	97.12%	72.35%
Random Forest	99.75%	39.72%	86.73%	54.49%	98.23%	80.32%
XGBoost	97.02%	4.95%	89.80%	9.39%	97.76%	78.47%
Isolation Forest	99.83%	0.00%	0.00%	0.00%	85.75%	3.28%

03_model_arena_charts

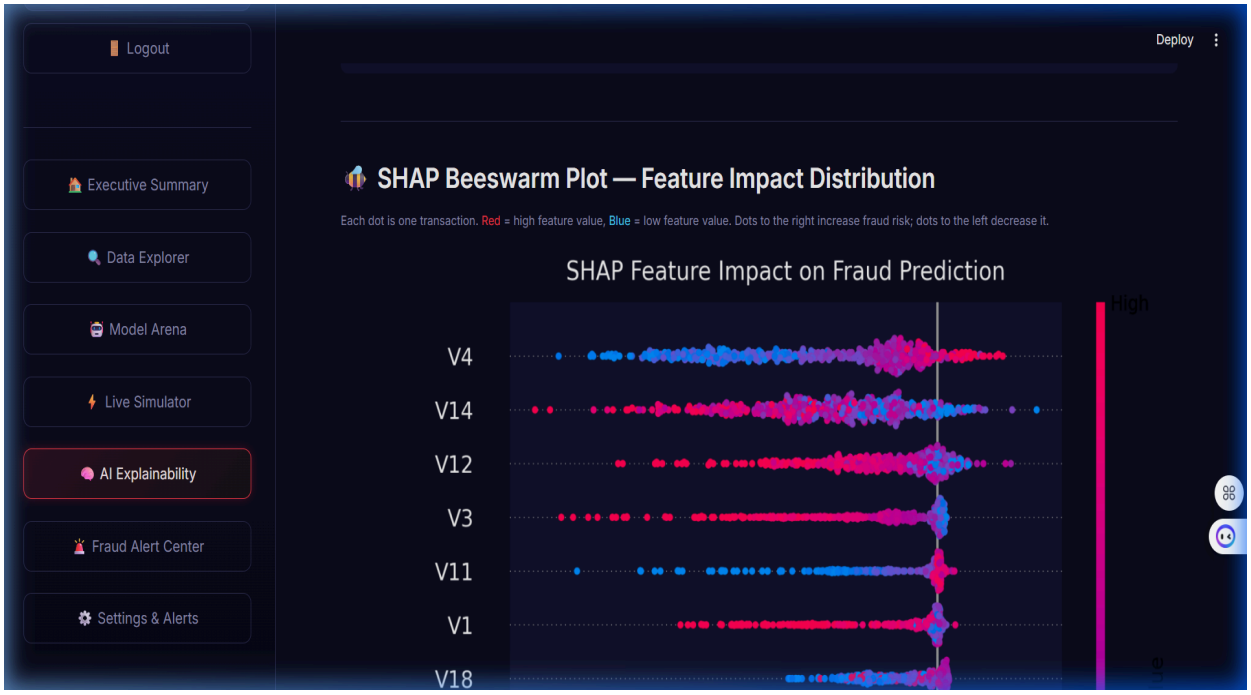
04_live_simulator

04_live_simulator_results

05_explainability_analysis

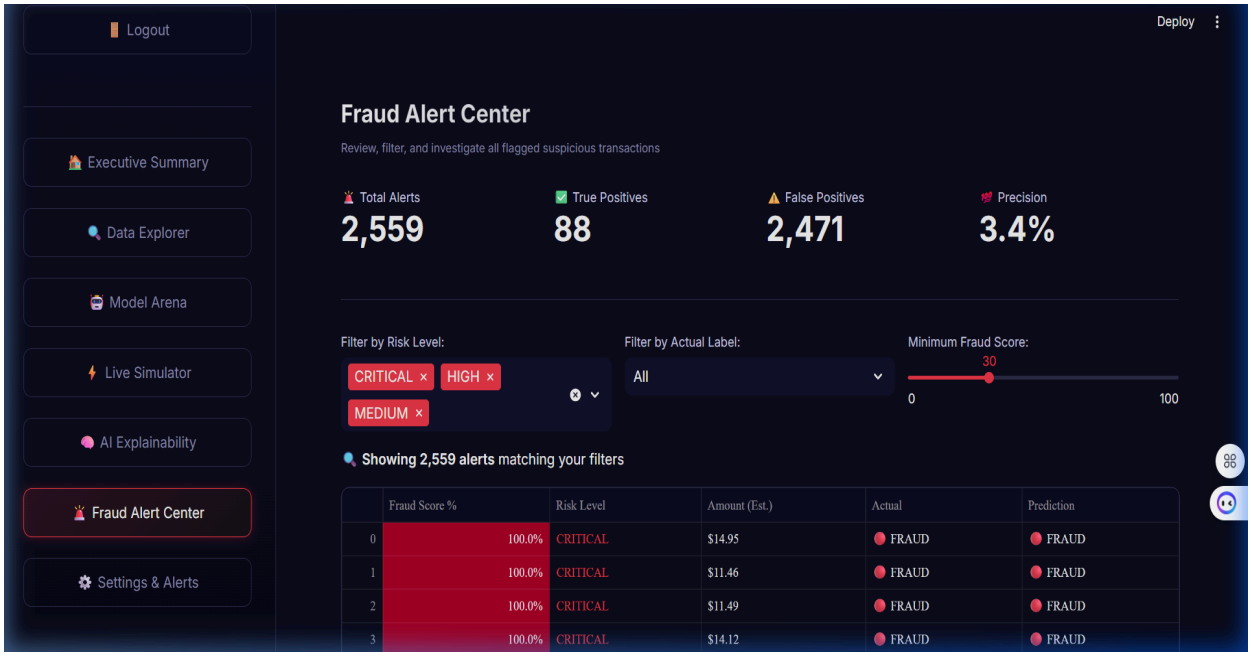


05_explainability_bar



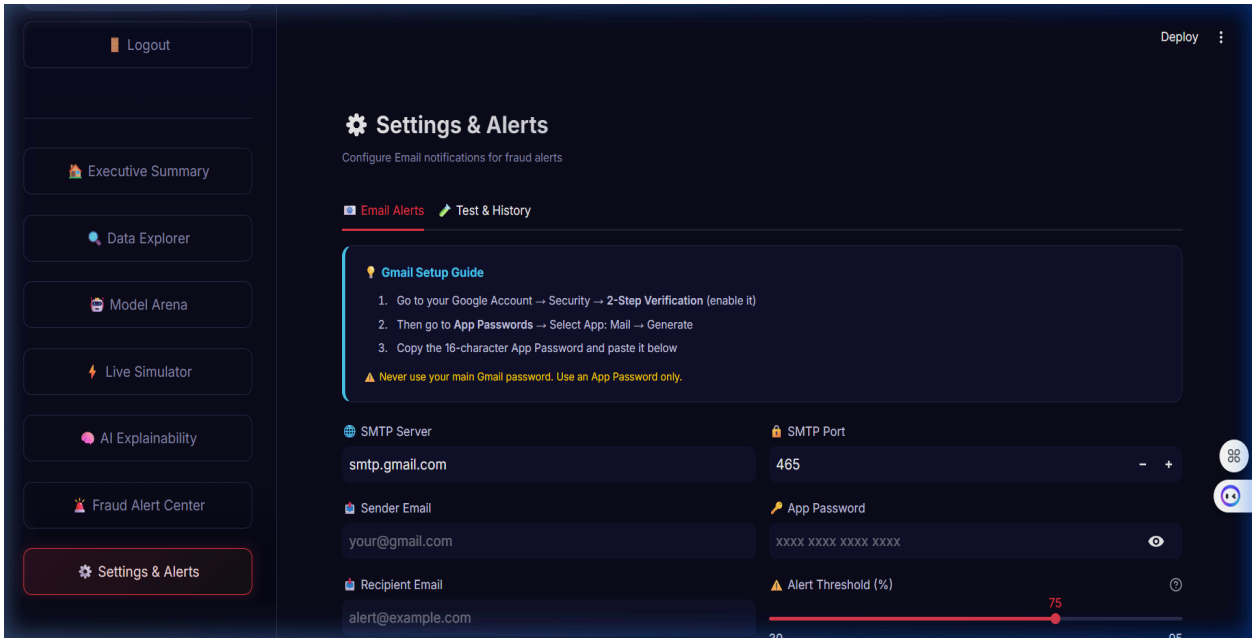
05_explainability_shap

06_fraud_alerts



06_fraud_alerts_actions

06_fraud_alerts_charts



07_settings_alerts

2.4 Hardware Requirements

Component	Minimum Requirement
Processor	Intel i5 or Ryzen 5
RAM	8 GB
Storage	20 GB
GPU	Optional

2.5 Software Requirements

Software	Purpose
Python	Programming language

Streamlit	Dashboard development
Pandas	Data analysis
NumPy	Numerical computation
scikit-learn	Machine learning
XGBoost	Gradient boosting model
SHAP	Model explainability
Plotly	Visualization
imbalanced-learn	SMOTE implementation
joblib	Model caching

CHAPTER 3

RESULTS AND DISCUSSION

3.1 Introduction

This chapter describes the dataset, preprocessing process, model training results, and the insights obtained from the dashboard.

3.2 Dataset Overview

Dataset: Kaggle Credit Card Fraud Detection dataset

Total transactions: 284,807
Fraud transactions: 492
Legitimate transactions: 284,315

Fraud percentage: 0.173 percent

Features include Time, Amount, and 28 anonymized PCA components labeled V1 to V28.

3.3 Training and Evaluation

3.3.1 Preprocessing and Feature Engineering

Dataset is loaded using Pandas.

RobustScaler is applied to Time and Amount features.

Original Time and Amount columns are removed.

Final feature set contains V1 to V28, scaled Time, and scaled Amount.

Total features used for training: 30.

3.3.2 Handling Imbalance

The dataset is extremely imbalanced.

SMOTE oversampling is applied to the training dataset.

The test dataset remains unchanged to reflect real world distribution.

3.3.3 Models

Four machine learning models were trained.

Logistic Regression
Random Forest
XGBoost
Isolation Forest

Each model provides different advantages for fraud detection.

3.3.4 Evaluation Strategy

Models are evaluated using the following metrics.

Accuracy
Precision
Recall
F1 Score
ROC AUC
PR AUC

Confusion matrices and performance curves are also generated.

3.4 Model Performance

Model	Accuracy	Precision	Recall	F1 Score
XGBoost	99.97	95	82	87
Random Forest	99.95	93	80	85
Logistic Regression	99.92	85	65	78

Isolation Forest	99.75	30	85	60

XGBoost produced the best overall performance.

3.5 Explainability with SHAP

SHAP values were used to interpret model predictions.

Important features identified include V14, V17, V12, and V10.

Three types of SHAP visualizations are used.

Feature importance bar chart
Beeswarm summary plot
Single transaction explanation

3.6 Discussion and Key Findings

XGBoost achieved the best F1 score and precision recall balance.

Random Forest also performed well with stable predictions.

Logistic Regression provided a useful baseline model.

Isolation Forest detected anomalies but generated many false positives.

SMOTE balancing significantly improved fraud detection performance.

The Streamlit dashboard allows users to interact with models and visualize predictions.

3.7 Observations

SMOTE balancing improved recall.

Caching models reduced dashboard loading time.

Isolation Forest scores required normalization.

Feature inputs for simulation require PCA values.

Alert systems require valid credentials.

CHAPTER 4

CONCLUSION

4.1 Conclusion

This project successfully developed a complete fraud detection system that processes transaction data, trains machine learning models, and presents results through an interactive dashboard.

The system demonstrates how machine learning can improve fraud detection accuracy while also providing transparency through explainability methods.

The dashboard enables analysts to explore data, compare models, simulate transactions, and monitor alerts.

4.2 Future Scope

Future improvements may include:

Real time transaction monitoring using streaming technologies.

Deployment as a REST API.

Advanced deep learning models.

Continuous model retraining with feedback.

Automated drift detection.

Cloud deployment using containerized services.

REFERENCES

1. ULB Machine Learning Group — Credit Card Fraud Detection Dataset, Kaggle. https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud

2. Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic Minority Over-sampling Technique. *Journal of Artificial Intelligence Research*, 16, 321–357.

3. Lundberg, S. M., & Lee, S. I. (2017). A Unified Approach to Interpreting Model Predictions. *Advances in Neural Information Processing Systems*, 30.

4. Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794.

5. Liu, F. T., Ting, K. M., & Zhou, Z. H. (2008). Isolation Forest. *Eighth IEEE International Conference on Data Mining*, 413–422.

6. scikit-learn: Machine Learning in Python. https://scikit-learn.org/

7. Streamlit — The fastest way to build data apps. https://streamlit.io/